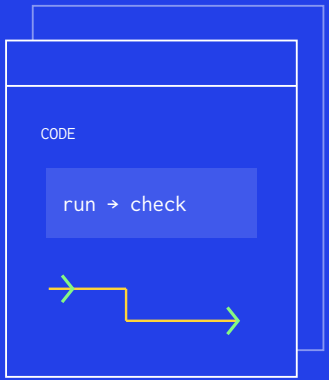
WEEK 3 / LECTURE 06

MLOps: Machine Learning Operations

Continuous Integration for ML Projects

How a change earns the right to merge: clean machines, honest checks, tests that can fail.



After this lecture you can

Each outcome ends in evidence you can show from your repository.

1. **Read a run**
   Say what a CI run proves, and what it cannot, from the commit it ran on.
2. **Design a pipeline**
   Order cheap checks first, give each job a reason, keep permissions minimal.
3. **Test for ML defects**
   Write regression tests that fail for named defects, and prove they can fail.

What did your Week 1–2 workflow really check?

Continuous integration only protects what it runs.

WHEN IT STARTS

Push to main
starts the workflow

Pull request into develop
starts nothing

ran never ran

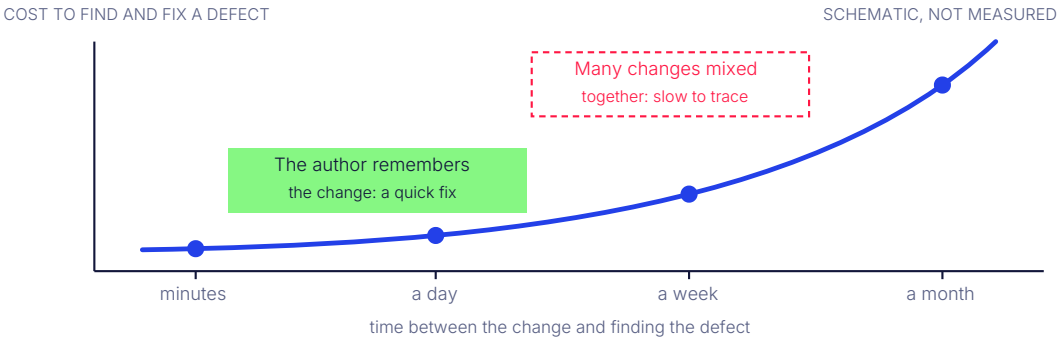
WHAT IT CHECKS

- Preflight
- Ruff style checks
- Infra tests
- Week 1 tests
- Exercise tests
- API tests
- Container build

Open your own ci.yml: which line decides each of these?

Why integrate continuously

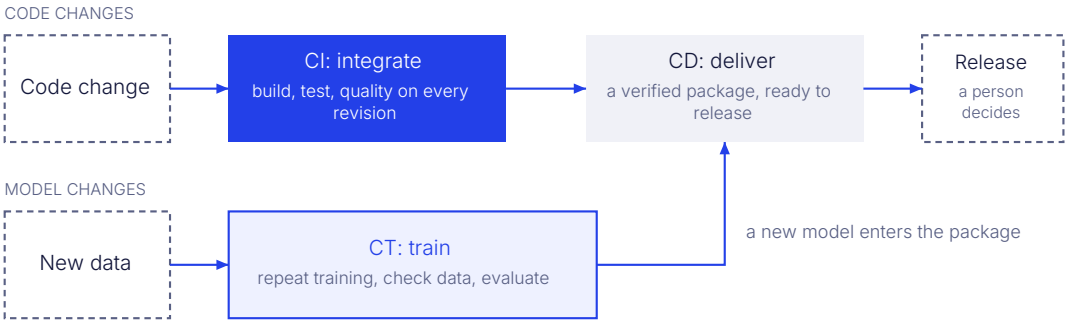
Small changes, checked quickly, are cheap to fix.



A defect found late costs more to find and to fix.

CI, CD and CT are three processes

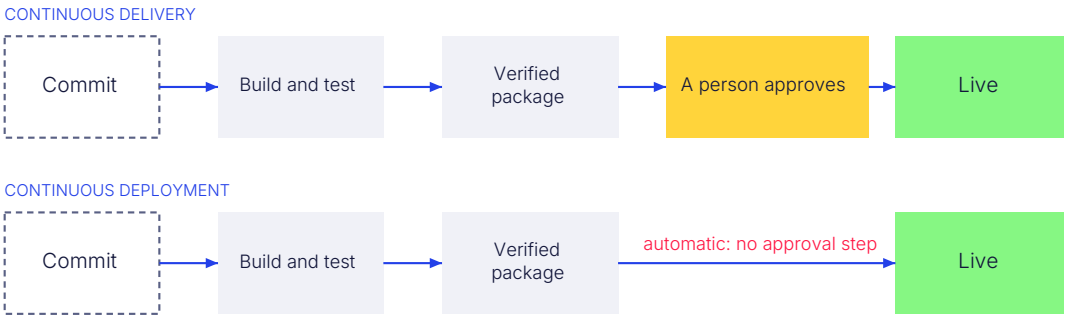
Related, but not three steps of one mandatory pipeline.



Today: CI. Lectures 7 and 8 add training gates and controlled releases.

Delivery or deployment? Name which one you mean

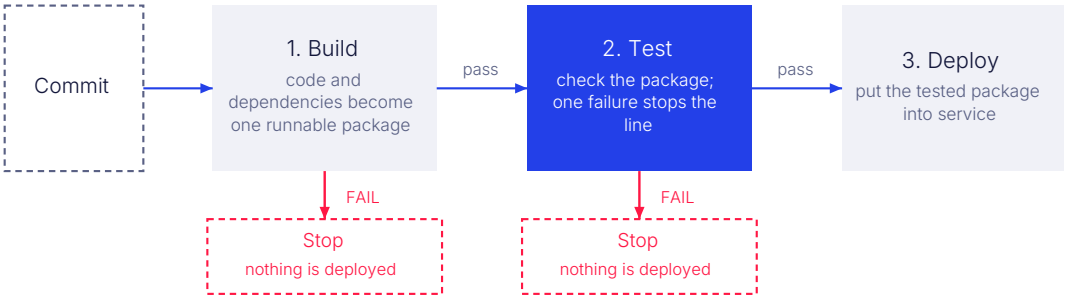
The two share the letters CD; they differ in who presses the last button.



Deployment is also the act of putting a version into service, in either approach.

A delivery pipeline has three stages

Build it, test it, then (only if it passed) deploy it.



Each stage runs only if the one before it passed. A pipeline can stop after any stage: not every project deploys on every change.

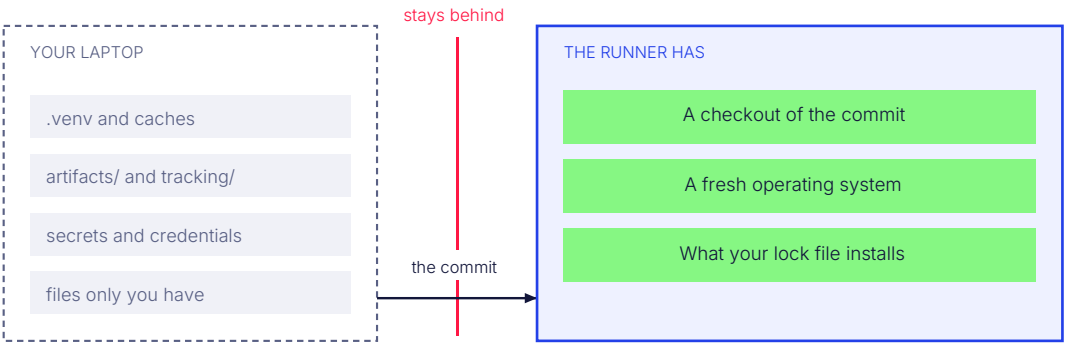
WHAT A GREEN RUN MEANS

Green means: this revision, this runner, these checks.

Not that the code is correct, and not that your laptop agrees.

A clean machine has none of your files

Know what the runner has before you trust a green run.



If a check passes only because of a local file, CI will tell you.

Local and CI evidence are different claims

Say which one you are citing.

YOUR LAPTOP

local evidence

- .venv and caches
- artifacts/ and tracking/
- secrets and credentials
- files only you have

≠

never mix

THE CI RUNNER

CI evidence

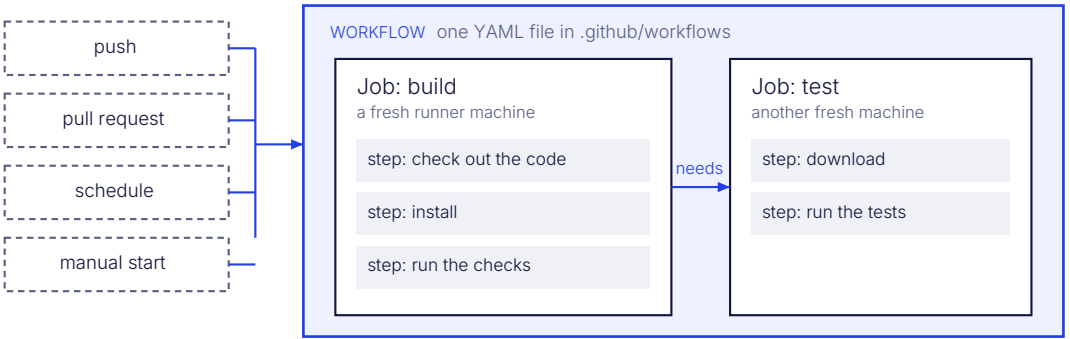
- a checkout of the commit
- a fresh operating system
- what the lock file installs

nothing else

Rule: never mix them. A result from one is not evidence for the other.

A workflow is events, jobs and steps

Each job starts from nothing; jobs share only what you hand over.



Nothing from your laptop is on the runner. Design question: what must each job be given?

Triggers decide what is protected

Ask the question in the last column for your own team.



`pull_request`

a pull request targets a listed branch

ASK

Which branches does your team merge into?



`push`

a commit lands on a listed branch

ASK

Is the integration branch checked after a merge?



`workflow_dispatch`

someone starts it by hand

ASK

What do you want to rerun on demand?



`schedule`

a cron time arrives

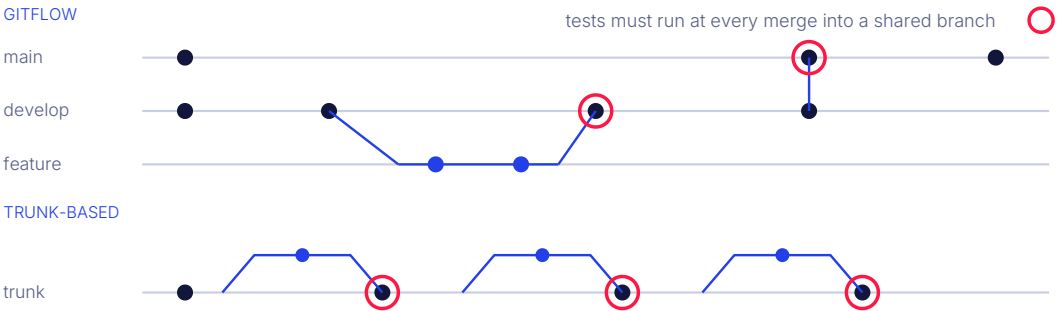
ASK

What should be checked when nobody pushes?

Gitflow has two integration branches. A filter that lists only one leaves the other unchecked.

The branching model decides where tests must run

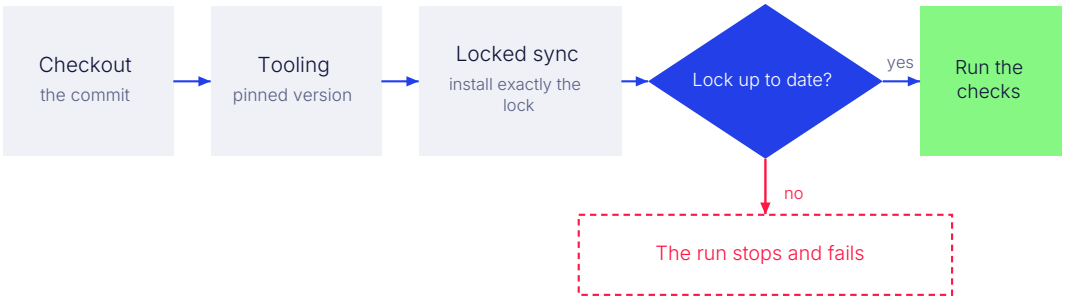
Testing matters in both; only the place to look changes.



A branch that nothing checks is a branch that can break unnoticed.

Install from the lock, on every run

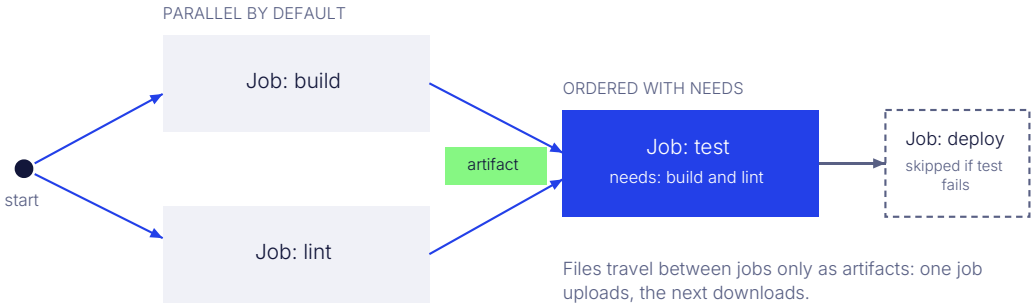
A machine enforces what the lock file promises.



A stale lock must stop the run. Never regenerate the lock to get green.

Jobs are independent unless you say otherwise

How one job hands work to the next is a design decision.



Design question: what must the next job be given?

Least privilege, pinned actions, time limits

Three habits your starter workflow already has.

illustrative workflow excerpt

```
permissions:
  contents: read
jobs:
  test:
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@a1b2c3d
```

MINIMAL PERMISSIONS
read-only is the safe default

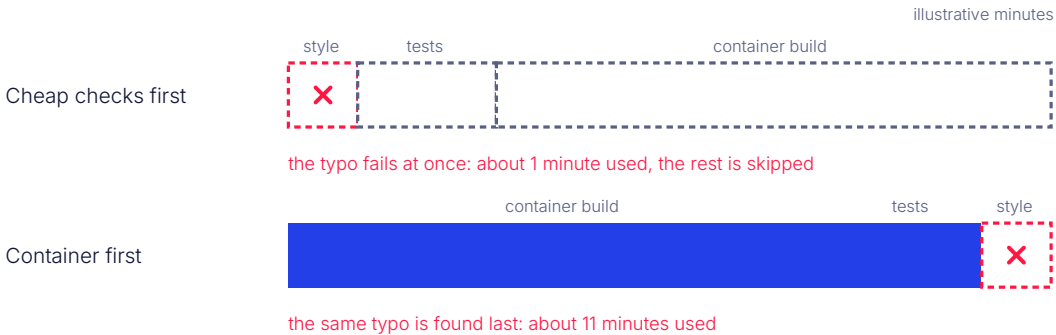
TIME LIMIT
a hung job stops, not your minutes

PIN WHAT YOU RUN
a tag can move; a commit SHA cannot

Keep all three when you extend the file.

Cost is a design input

Every run uses minutes, and private repositories draw on a monthly allowance.



Fast feedback is also a cost: slow pipelines get ignored.

“No run appeared”: four places to look

Most missing runs are one of these.

LOCATION

`docs/ci.yml` ✗

`.github/workflows/ci.yml` ✓

it must live at the repository root

TRIGGER

```
branches: [main]
```

a pull request into develop: no run

does the filter include the target branch?

SYNTAX

```
steps:
  - run: pytest
   - run: pytest
```

a YAML error: the workflow is marked invalid

SWITCH

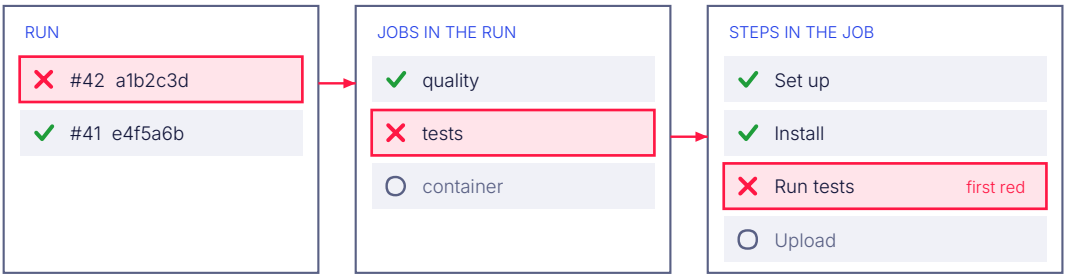
Actions is disabled

for the repository or for the workflow

Fix one at a time and read the Actions tab after each push.

Reading the Checks tab

Three levels: run, job, step. Start at the first red one.



one per event, one commit

one per machine

one per command

Save what you read as text; the page will not last.

DESIGN RULE

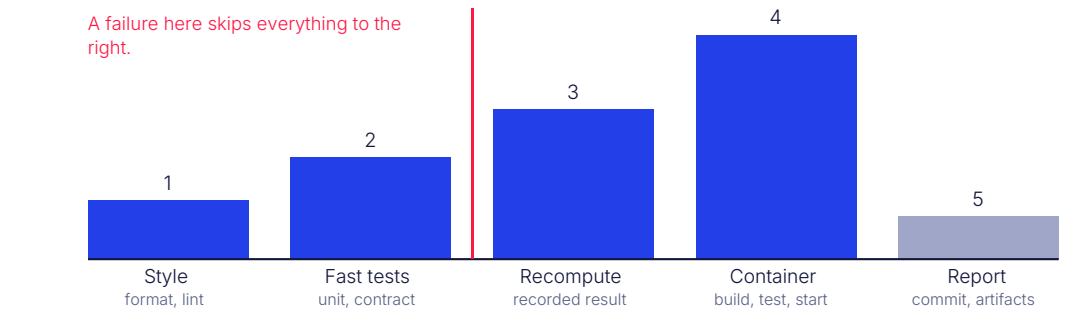
If CI can run it,
you can run it locally.

Use the same locked command in the workflow and in your terminal.

Order the stages by cost and purpose

Fail fast: a typo should not cost a container build.

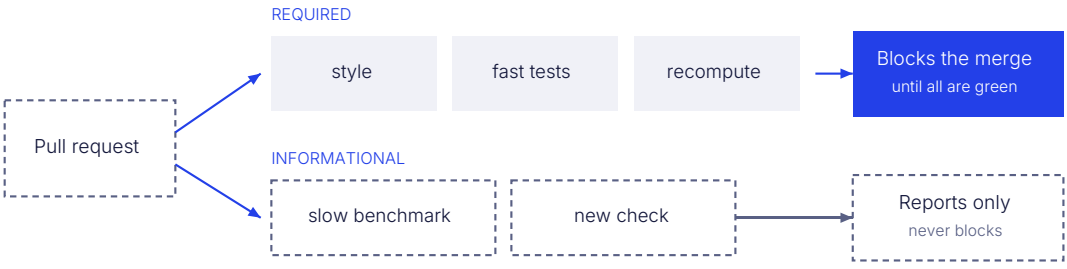
COST AND TIME (SCHEMATIC)



An example order from a spam-filter project; your project may need another.

Decide which checks block a merge

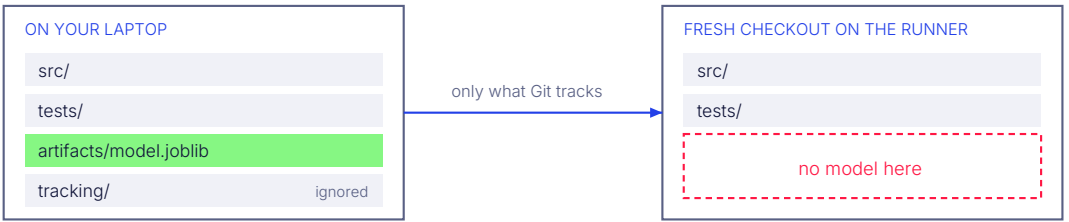
Block only on checks that are fast, stable and meaningful.



On free private repositories GitHub cannot block the merge button for you; your team agreement does.

A fresh checkout is empty of models

Ignored folders are not in Git, so the runner cannot see them.



TWO WAYS TO GET A MODEL

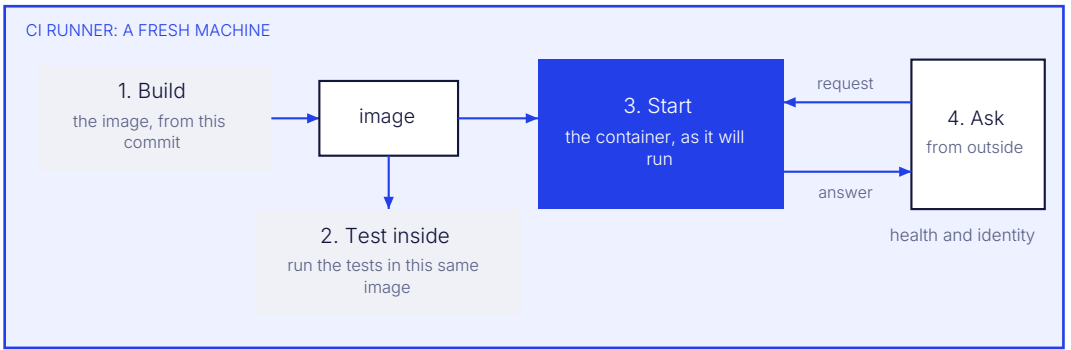
Build it on the runner
the run made it, so it is CI evidence

Use a small stand-in
a smaller claim: say what it proves

Which option fits which claim is your decision.

Prove it where it will run

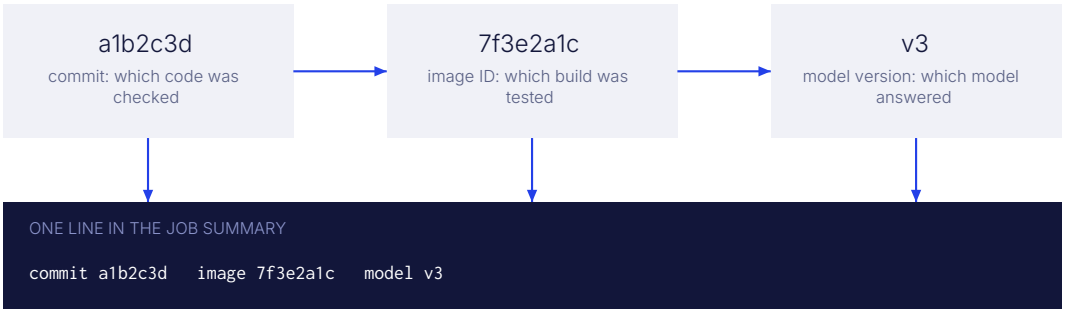
A container job checks the thing you would ship.



The run should print the commit and the image it checked.

Make the run say what it built

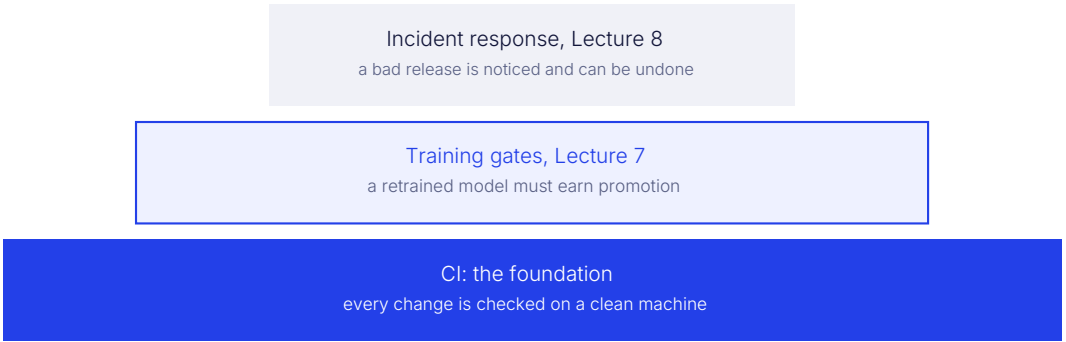
Evidence must name its subject.



A job summary or a log line is enough; a screenshot alone does not connect them.

What comes after CI

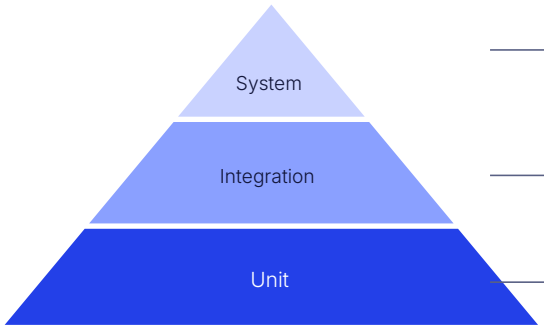
Each lecture adds one kind of control.



CI is the foundation: later controls assume trustworthy checks.

Tests come in levels, and a good test is small

Examples come from a spam filter; yours will differ.



System: few, slow, broad
a request to the running API returns a valid answer

Integration: parts together
the cleaner and the model return a score for raw text

Unit: many, fast, narrow
the text cleaner lowercases and strips markup

A good test arranges its input, acts once, asserts the result, and does not depend on other tests.

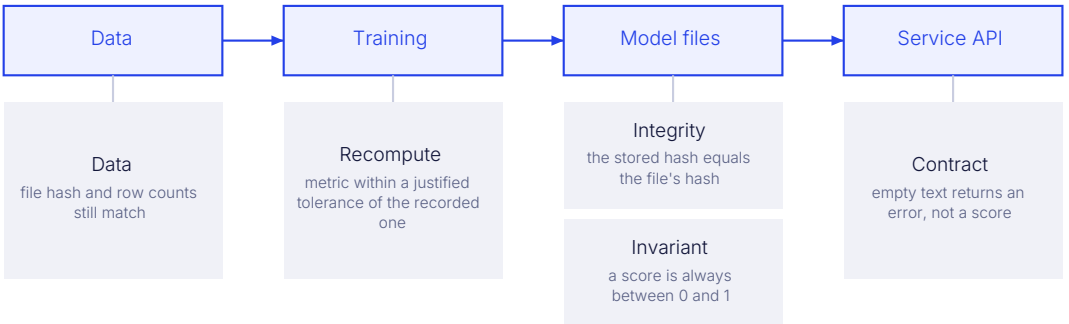
A regression test protects what already works

It exists so that a future change cannot quietly undo it.

1. **Pin a behaviour**
   Choose something that worked and must keep working.
2. **Name the defect**
   State which mistake would make the test fail.
3. **Keep it stable**
   Same input, same answer: no hidden dependence on time, order or luck.

Kinds of regression tests for ML systems

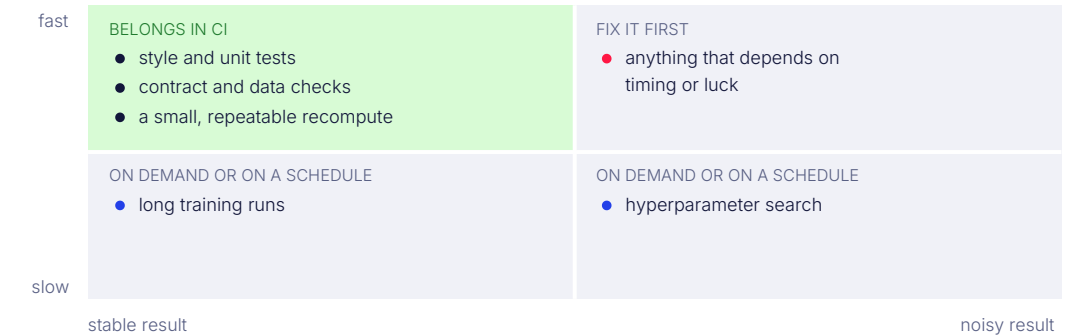
Examples come from a spam filter; yours will differ.



A bug-fix test is a regression test written on the day you fix the bug.

What belongs in CI, and what does not

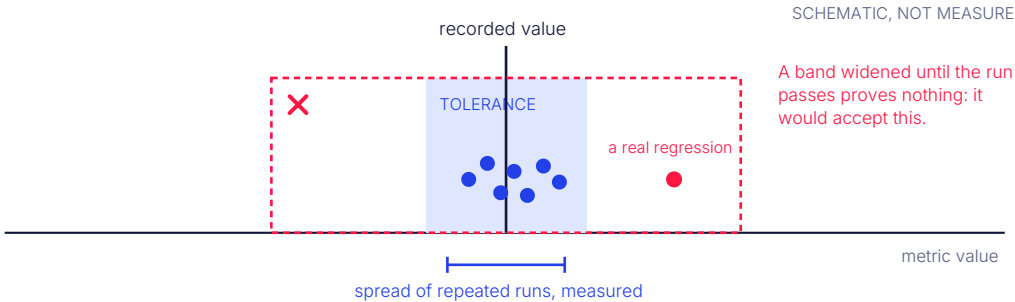
CI is for fast, repeatable, meaningful checks.



Heavy work can run on demand or on a schedule instead.

A recomputed result needs a tolerance

Numbers from two machines are rarely identical to the last digit.



Changing a tolerance is a reviewed change with a reason.

Flaky tests are defects too

A test that fails at random teaches the team to ignore red.

SAME COMMIT, DIFFERENT RESULTS



WHERE RANDOMNESS HIDES

- time and time zones
- test order, shared state
- random seeds, threads
- network calls

WHAT TO DO

Remove the cause
fix seeds, isolate state

Quarantine
only with an owner and a date

Retry until green ✗
never: it hides real failures

A flaky check is worse than no check: it hides real failures.

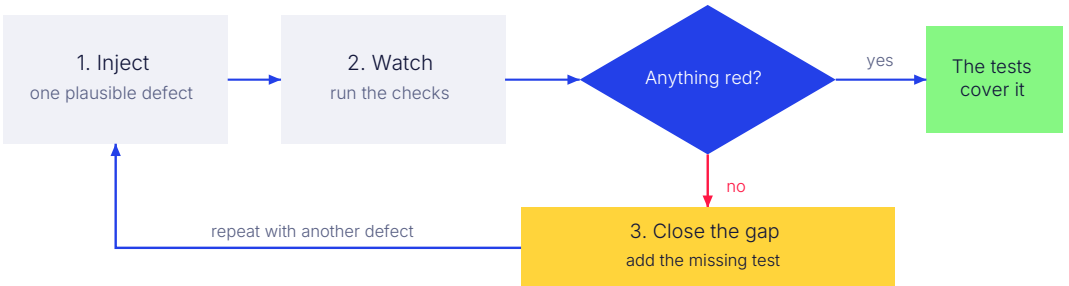
TESTS THAT CANNOT FAIL

A test that cannot fail proves nothing.

Ask of every test: which defect would turn this red?

Break it on purpose to learn what is covered

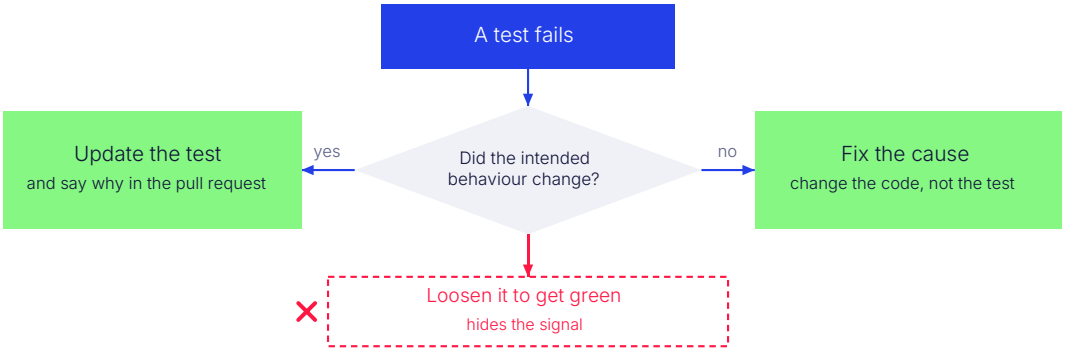
Mutation thinking: change the code and see who notices.



If everything stays green, the pipeline has a gap. Do this on a throwaway branch or in a pull request you do not merge.

Good and bad reasons to change a test

A test is a statement about intended behaviour.



Every change to a test deserves the same review as a change to code.

Read a failure

Work from the first red step upwards.

JOB STEPS

- ✓ Set up job
- ✓ Check out
- ✓ Install
- ✗ Run tests
- ○ Upload report

1

LOG OF THE RED STEP

```
loading data ... ok
training model ... ok
computing score ... 0.43
FAILED test_score
AssertionError: score too low
```

2

3

1 start at the first red step 2 the cause is above it 3 name it before changing code

Do not rerun until it passes: that hides flakiness. Write the cause in your own words before you touch the code.

Reproduce with the same command

If you can run it locally, you can debug it.

ILLUSTRATIVE EXAMPLE

Copy the failing step's command from the log and run it from a clean environment, as the runner would.



Passes locally, fails in CI? The difference is the finding: files, versions, order.

Repair, do not mute

Each shortcut removes the signal the pipeline exists to give.

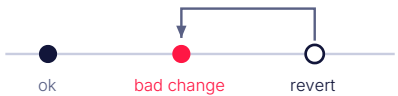


If you must change a check, change it in a pull request and say why.

Revert or fix forward?

Both are fine; the history should show which one you chose and why.

Revert



Fix forward



REVERT WHEN

- the change is large or unclear
- the branch is blocked for others
- you need green now

FIX FORWARD WHEN

- the cause is understood
- the fix is small
- the history stays readable

Either way the red run and its repair stay in the history.

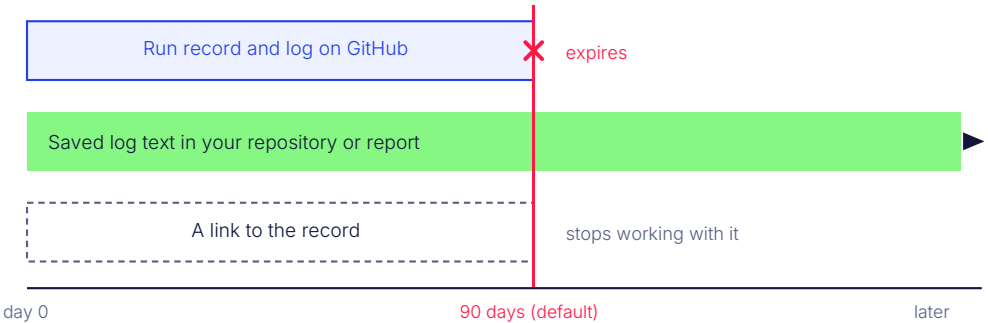
THE LIMIT OF CI

Green is evidence, not proof.

CI shows that these checks passed on this commit. Whether they were the right checks is still your job.

Evidence that survives

Run pages are temporary; text you saved is not.



Check your repository's retention setting if you need longer.

Three ideas worth keeping

If you remember only these, you can rebuild the rest.

1 **Green has a scope**

This revision, this runner, these checks: nothing more is proved.

2 **A test is worth what it can catch**

Name the defect, then show that the test can fail.

3 **Evidence must survive**

Save the text, name the commit, say who produced it.

Where the ideas come from

Public documentation and well-known practice; read the originals for every option.

REFERENCES

Examples on slides are illustrative. Run them in your own project before you rely on them.



Check each source's current version before relying on it.

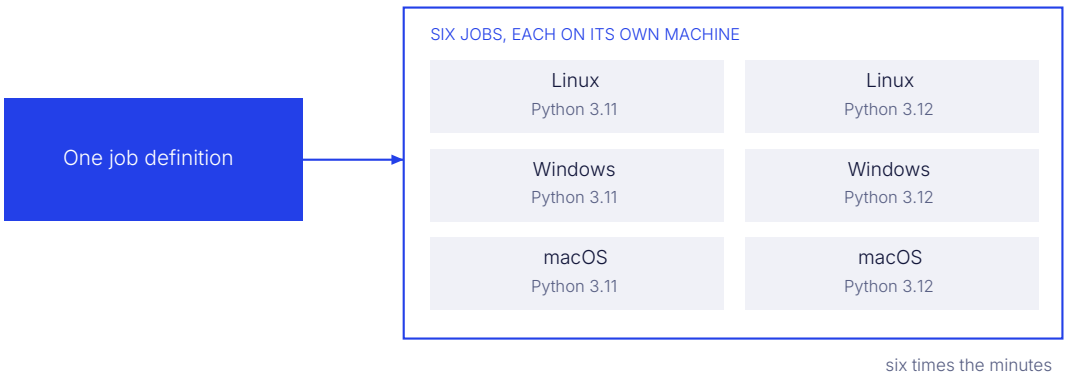
OPTIONAL APPENDIX

Go deeper, when you need it.

These slides extend the lecture. None is needed for the core ideas.

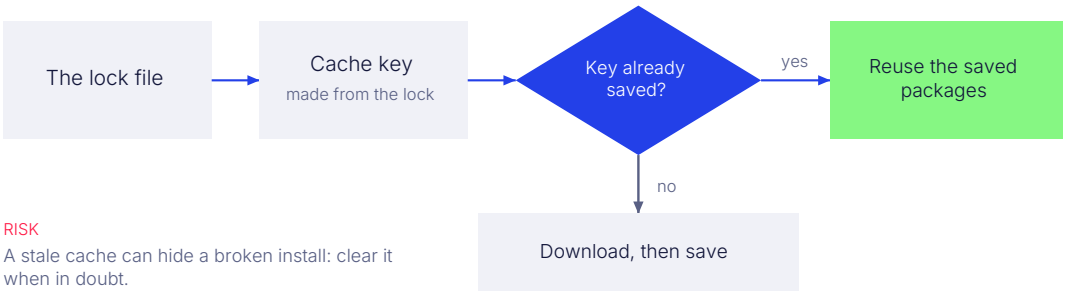
Matrix builds

Run the same job across several versions or systems.



Caching dependencies

Reuse downloads between runs of the same lock.



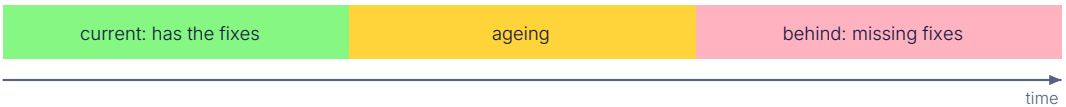
RISK

A stale cache can hide a broken install: clear it when in doubt.

Keeping pins up to date

A pin protects you until it falls behind.

A PIN PROTECTS YOU, THEN AGES



1. Bot
finds a new version

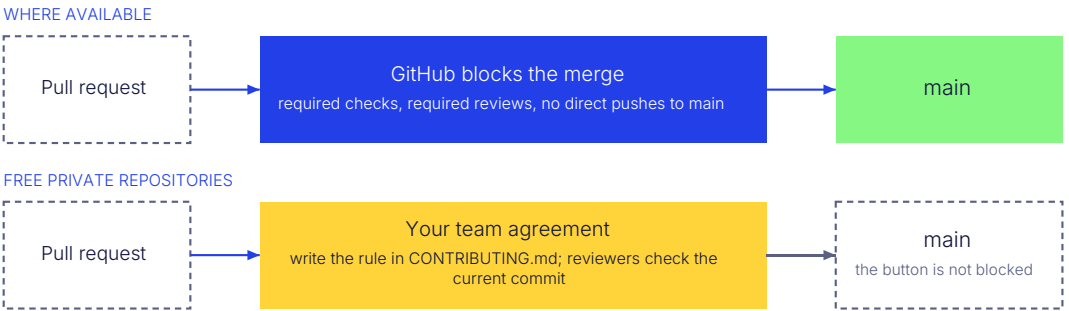
2. Pull request
opened by the bot

3. Your checks
run on that pull request

4. You review
passing is a candidate, not an approval

Branch protection and rulesets

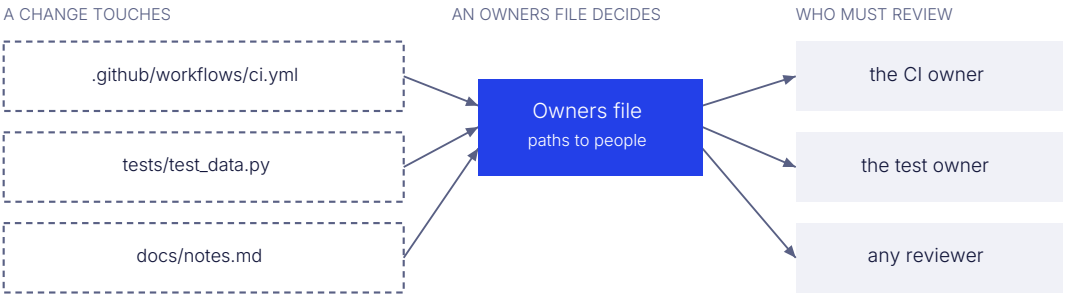
Where your plan allows it, GitHub can require checks before a merge.



Public repositories on a free plan can use protected branches; check your own plan.

Routing reviews

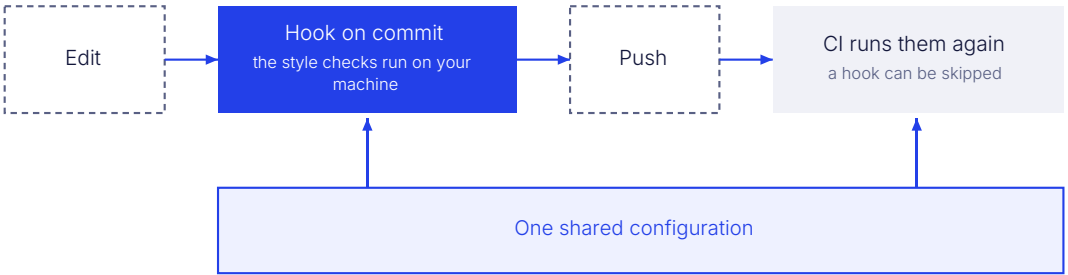
Make the right person see the right change.



A pull request template reminds the author to attach evidence. Rotate reviewers so knowledge does not sit with one person.

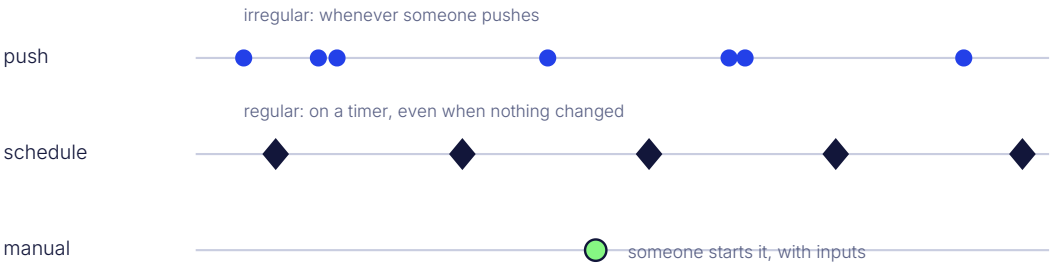
Pre-commit: the same checks, earlier

A hook catches the easy problems before they reach CI.



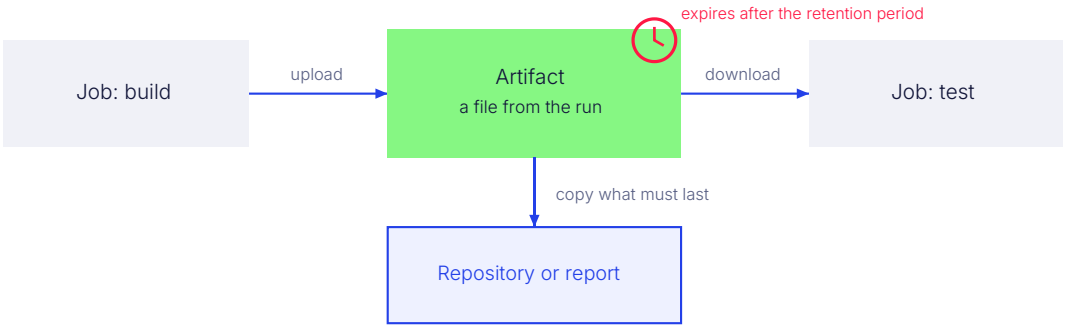
Schedules and manual runs

Not every check starts with a push.



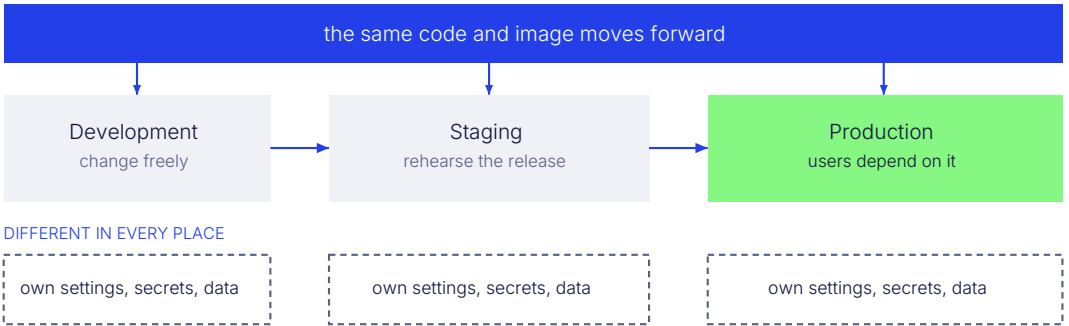
Artifacts and retention

Files a run produces, and how long they last.



Environments as stages of confidence

A change is promoted through places that look more and more like production.



This course has local runs, CI runs and the final release; the idea is the same.

Infrastructure as code

Describe the environment in files, not in clicks.



reviewed, versioned, repeatable

Online testing: A/B, canary, shadow

These reduce the risk of a release; Lecture 8 returns to them.

